Convolution Neural Networks for Edge Devices

Advanced Topics in Image Processing, Griffith University

Nathan Burg
Griffith University
s5348935

Abstract

1 Introduction

The domain of digital image processing is among the most diverse in all of computer science, with complex subdomains like compression, generation, and classification. The task of image classification is a unique problem as it is the only task coupled to and directly measured against human perception of images. This is leveraged heavily in user-facing cyber security, in systems like CAPTCHA, where human visual processing cannot be trivially replicated by malicious software—demonstrably, computers are an exceptionally poor analogue of human perception.

Image classification is a domain of machine learning predicated on the 'feature space' of an image. Feature extraction can be understood as a bridge between the suitable representation for human vision, and a representation that is statistically separable between classes. Machine learning classification algorithms exploit this statistical separability using numerical optimisation algorithms, to effectively 'learn' the features associated with a class.

Classical machine learning techniques like the Random Forest, k-Nearest Neighbours, and Support Vector Machine algorithms rely extensively on careful feature extraction. These algorithms utilise high-dimensional vector representations of an image to classify according to relative alignment in feature space, where the vector is extracted from statistical analysis of an image. Manipulation techniques like convolution serve to make statistical features of an image—such as object edges—more pronounced.

Artificial Neural Networks (ANNs) and the Multi-Layer Perceptron (MLP) are statistical fitting structures that identify common traits between samples of a class using cross-entropy loss and backpropagation. In these simplified models, a 1-dimensional input vector of a feature space is propagated through a series of layers, to then select a class from a 1-dimensional output vector using `softmax` or similar. This is the foundation of all modern machine learning.

Convolutional Neural Networks (CNNs) extensively serve to automate the feature extraction process through deep learning. For small images with high signal-to-noise ratios (SNR) such as MNIST, an MLP is generally sufficient through one-hot encoding of the image itself. For larger images, feature extraction is necessary to identify isolated patterns and improve SNR. CNNs use numerical optimisation to locate the set of convolution kernels that produce the correct feature space, and then use an MLP to classify them, as a two-step machine learning process.

The training and inference process of CNNs are generally very computationally expensive, and require expensive GPU hardware to deploy practically. Vendors of these devices each provide their own proprietary solution to General-Purpose GPU programming (GPGPU), and in the specific case of machine learning, there is an industry-standard vendor lock on the NVIDIA CUDA ecosystem.

We propose an experimental solution that relies on Vulkan, which is a computer graphics API that can be leveraged for GPGPU programming through headless 'compute shaders.' Vulkan is designed for maximum hardware compatability, but also exposes the full functionality of various GPU architectures through a rich set of extensions. Vulkan compatibility extends beyond consumer desktop GPUs, and also includes mobile phone chipsets, edge devices like the NVIDIA Jetson, and single-board computers like the Raspberry Pi. Our goal is to deliver a prototype of a complete GPU machine learning pipeline for Convolutional Neural Networks using the Vulkan API, and demonstrate the engineering challenges of developing such a pipeline under significant time constraints.

2 Literature Review

2.1 Image Classification

In order to approximate the learning effects of human vision, algorithmic image classification generally requires a large repository of sample images. This repository must be labeled with a ground-truth baseline, which establishes the underlying principle of machine learning and image classification. Principally, these techniques

do not have the same subjective visual understanding as humans, but instead use statistical approximations of what feature vector is most geometrically similar to manually labeled target.

Image classification is entirely predicated on clean repositories of images, which itself is subject to significant quality control issues. Using algorithms to classify images constitutes a substantial portion of the literature, but it is also important to consider the difficult problem of data collection and labeling, and the importance of clean data particularly in the area of highly expressive machine learning models.

A complete image classification pipeline has three core components, being the image repository for training, the feature extraction algorithm, and the feature space classifier. This specific coupling is the general foundation of all classification pipelines in machine learning.

2.1.1 Image Repositories

As image classification as a concept is almost entirely predicated on high-quality training data, images should be of consistent quality across classes, correctly labeled, and ideally of high signal-to-noise ratio to prevent shortcut learning. A low-quality dataset leads to poor generalisation in trained models—because many modern training sets are effectively 'black boxes' in terms of content with image counts in the millions, consistent methodological rigour of collection and labeling should be practiced.

The MNIST dataset is common for simple image classification tasks, and consists of 60000 greyscale images at a resolution of 28×28. The subject of the images is handwritten numerical digits. The simplicity of the images in this dataset permit usage of the image itself as a feature vector—as the input size of 784 is small enough to be tractable—making MNIST a common target for MLP-based classifiers.

The CIFAR family of datasets, including both CIFAR10 and CIFAR100, are more complex images that are harder to separate in feature space than MNIST. CIFAR images are 32×32 RGB depicting real subjects across several mutually exclusive categories, such as dog, deer, truck and ship. As this dataset consists of RGB images, the size of the input vector increases dramatically to 3072, which significantly reduces the SNR in feature space. It is generally more common for CIFAR classification to be solved with convolution-based networks, as the 3 channels represent a tensor that maps directly to CNN architecture, and the feature extraction of the convolution layers can more effectively filter noise.

ImageNet is considered the standard dataset in computer vision research. The complete set contains over 14 million images across more than 20000 classes at varied image resolutions. An extensive preprocessing pipeline is needed for this dataset to normalise image resolution to a standard square appropriate for standardised feature extraction. A variant of ImageNet called ImageNet-1k or ILSVRC2017 is very common in model benchmarks, where breakthrough models like AlexNet and VGG were developed and deployed specifically to classify on this dataset.

2.1.2 Feature Extraction

Feature extraction is fundamentally a signal processing problem. Images are easily identifiable to humans, and perhaps counterintuitively the visual signatures that allow this recognition are effectively noise in terms of algorithmic processing. Feature extraction maps high-noise image data into high-signal 'hints' about a certain feature of the image, which can include edge detection, colour histograms, or frequency-space transforms. The primary objective of feature extraction is to reduce the high-frequency image into a statistically separable feature vector.

Earliest feature extraction techniques, such as Otsu's thresholding, use a colour histogram for the image to apply a binary threshold and separate a subject from the background. The Sobel filter and Canny edge detection use convolution filters to isolate object edges. As examples of feature extraction, these techniques correctly amplify the signal of desired features, but do not reduce the dimensionality of feature space.

Advanced techniques like Histogram of Oriented Gradients (HOG) and Scale-Invariant Feature Transforms (SIFT) apply several convolution filters in sequence to extract localised colour histograms for discrete segmentations of an image. These techniques differ in the shape of the output data, which governs its utility—SIFT produces a list of points-of-interest for an image, while HOG produces a fixed-dimension distribution of edge normal vectors. The feature vector of HOG is invariant while SIFT produces a list of relevant edges. For HOG specifically, this feature space is highly separable using machine learning, and is very useful for classification problems.

2.1.3 Machine Learning

Machine learning is applied numerical optimisation for statistical prediction problems. For image classification, a model must be optimised to separate classes using geometric and statistical difference between feature space vectors. This can be achieved with modern deep learning networks, but algebraic or probabilistic techniques—often called 'classical machine learning'—are also highly effective and generally more computationally efficient for classification of highly separable feature vectors. The Support Vector Machine (SVM) is an example of classical

machine learning, which treats a feature vector as a high-dimensional geometric object and classification as a convex quadratic optimisation problem. The output of an SVM is a max-margin hyperplane for the decision boundary, which is trivial to compare during classification.

Random Forest (RF) classifiers observe the feature vector as a set of subspace elements. Decision trees for an input set are formed from localised subsets of the feature vector, with an independent tree for each. During classification, an input feature vector is segmented and evaluatied for each sub-feature, and the final output is the majority decision of each sub-feature decision tree.

K-Nearest Neighbours (kNN) is an instance-based algorithm that classifies images by geometric distance in feature space. Being instance-based, kNN has no training process, and trivially returns multiple images that are 'most similar' to the input feature vector. The data stored for this algorithm is far higher than SVM or RF, as each training image is retained and ranked lazily. The benefit of this is total dynamic classification where images can be added or removed individually, which is impossible in SVM and RF.

Each of these methods, while effective classifiers, rely on statistically effective feature extraction, which is an unsolved problem and is not consistent across domains. Modern techniques like Convolutional Neural Networks implicitly couple feature extraction with classification, forming a complete classification solution. Consequently, classical machine learning is less utilised in modern image classification literature.

2.2 Deep Learning

Deep learning and neural networks are a class of statistical prediction models that encode arbitrary nonlinear functions in high-dimensional latent space. The model itself is constructed of several layers of high-dimensional vectors called 'layers'. Propagating an input vector through these layers involves two steps—the transformation, and the activation. The first stage applies the model 'weights', which are a linear transformation in feature space that weight features toward an output signal. The activation function then applies a nonlinear transform, such as ReLU, to effectively distort the linear input signal into nonlinear feature space. In the final layer, the `softmax` function is applied to identify the predicted class using the non-normalised 'logits.'

The training process of a neural network is first predicated on a loss function, which effectively represents the magnitude of misprediction by the neural network, which becomes a gradient in vector space. This vector is then propagated backwards through the network and the layer weights are updated according to this gradient. Multiple functions must be simultaneously approximated in neural network training, so the maximum difference or 'learning rate' for this weight perturbation is vanishingly small. The desired outcome is slow convergence toward an approximate function that can classify data not included in the training set.

2.2.1 Multi-Layer Perceptron

The Multi-Layer Perceptron (MLP) is a standard formulation of a fully-connected neural network. This form operates exclusively on one-dimensional input and output vectors, which poses structural limitations for image classification problems. Despite retaining the complete input signal, there is no consideration of spatial coupling inherent to the architecture, meaning the topology of the image is destroyed—this also means manual feature extraction of the image must be performed. While an MLP can effectively classify a feature vector representation of an image, classical machine learning techniques can do so more efficiently. This directly motivates a coupling of feature extraction and classification for deep learning image classification to be tractable.

2.2.2 Convolutional Neural Networks

Convolutional Neural Networks extend MLPs by propagating layers in higher dimensions. CNNs are particularly suited to image recognition, as they can be understood as a unified pipeline of both feature extraction and classification. This is achieved by representing the early layers as 3D tensors instead of 1D vectors, encoding both spatial information and colour channel information. The linear transform is then a convolution for each element of this tensor, where the convolution kernel is the weight for that layer. This tensor is then downsampled along its spatial dimensions, and subsequent convolutional layers utilise expanded kernel dimensions to increase channel depth. This repeats until the tensor is near-1-dimensional, where it then becomes the input for an MLP-like classifier.

LeNet-5 The original formulation of the Convolutional Neural Network was designed for handwritten digit and symbol classification, specifically motivated by the topological collapse of MLPs for this problem. To preserve spatial structure, LeNet introduced the structural template of modern CNN architectures by stacking successive convolution and sub-sampling blocks that terminate to fully connected layers. While effective for low-resolution digit recognition compared to an MLP, the architecture lacked the computational scaling capacity and depth required to process and classify more complex images.

AlexNet This formulation optimises the basic CNN by introducing the Rectified Linear Unit (ReLU) to CNN architectures. The ReLU diverges from the standard artificial neuron design, which traditionally relies on sigmoid or `tanh` activation producing normalised values between 0 and 1. ReLU reduced training time on CIFAR-10 by about 85%, allowing the model to scale to larger input tensors, and more images. Consequently, AlexNet superseded all available models at the time until sufficient GPU hardware development permitted more advanced formulations.

VGG The VGG architecture investigates the effect of CNN network depth on classification performance. By using a uniform convolution kernel size—the smallest possible at 3×3—and adding more convolution layers, VGG introduces stacked convolution and the effective receptive field. This homogeneous stacking yields both increased nonlinearity through additional ReLU activations, and reduced the overall parameter count relative to larger single-layer filters. The VGG-16 formulation, applied to the ImageNet dataset, supplanted AlexNet in classification accuracy.

Structurally, VGG is limited by its computational cost. The additional layers substantially increase training time and memory overhead, making it impractical in many applications. Furthermore, increasing the model depth even further has a boundary where performance begins to degrade, which is attributed to the vanishing gradient problem. As a gradient passed through backward activations in training, the gradient becomes smaller with each layer, where it is eventually extinguished by quantisation error in floating point division.

ResNet While VGG established that architectural depth directly correlates to feature abstraction performance, the vanishing gradient problem cannot be suppressed by conventional CNN architectures. The Residual Network (ResNet) was introduced to redefine the learning objective of deep convolutional layers by introducing identity skip connections that allow the layers to learn a residual mapping of activation outputs. Rather than forcing a sequence of layers to approximate an absolute mapping $H(x)$, ResNet structures the convolutional block to learn a residual mapping, formalised as $F(x) = H(x) - x$. The original input tensor x bypasses the weight layers entirely through a shortcut path and is recombined through element-wise addition. This yields the final block activation:

$$H(x) = F(x) + x \tag{1}$$

This modification fundamentally alters the mechanics of backpropagation. When computing the block activation derivative with respect to the input the chain rule yields:

$$\frac{\partial H(x)}{\partial x} = \frac{\partial F(x)}{\partial x} + 1 \tag{2}$$

The constant +1 term ensures that even if the gradient through the convolutional layers ($\frac{\partial F(x)}{\partial x}$) approaches zero, the error signal can propagate backward across the addition operator completely unobstructed. By establishing these checkpoints for gradient flow, ResNet effectively mitigates gradient attenuation, enabling the stable convergence and higher accuracy of deep networks (such as ResNet-50 and ResNet-152) that far surpass the depth limitations of VGG.

2.3 Edge Computing

2.3.1 Definition and Application

Edge computing in the context of computer vision is the migration of compute workloads from centralised cloud servers directly to the physical periphery where data is collected. For many applications, connection stability, latency constraints, or infrastructure may limit the availability and utility of centralised computing.

This environment is constrained primarily by Size, Weight, Power and Cost (SWaP-C). Edge topologies typically operate on constrained power budgets and possess limited thermal dissipation capabilities, making them hostile to the massive parameter counts and floating-point operations inherent to state-of-the-art CNNs. Consequently, deploying deep learning models to the edge necessitates a strict focus on compute efficiency, architectural compression, and highly optimised hardware execution.

2.3.2 Vulkan

Vulkan is a modern graphics API designed to operate on low-level GPU primitives, which has the secondary benefit of near-universal GPU compatibility. A common usage in General-Purpose GPU (GPGPU) programming is cross-platform massively-parallel computation using Vulkan—unlike previous abstractions like OpenGL ES, Vulkan directly enables arbitrary command buffer submission for headless execution of user-defined compute shaders. Several frameworks currently exist for cross-platform machine learning using Vulkan, including backends of bleeding edge libraries like `llama.cpp`, Vulkan Kompute, and NCNN.

The structural significance of Vulkan is threefold:

1. For high-powered devices, Vulkan enables precise, low-level GPU control that can match or exceed the performance of industry-standard, high-level abstractions. Device specific features are supported through Vulkan extensions to push performance even further.
2. The proprietary vendor lock-in of NVIDIA's CUDA ecosystem is completely bypassed, permitting efficient parallel execution across diverse hardware architectures
3. An overwhelming majority of modern edge devices, including smartphones and single-board computers, feature high-performance Vulkan drivers capable of executing pre-compiled SPIR-V bytecode with deterministic precision.

3 Methodology

The primary artefact of this paper is an end-to-end Vulkan compute and machine learning runtime, with a structurally complete implementation of the VGG convolutional neural network architecture. This software is composed of several layers of abstraction for practical extensibility.

3.1 Vulkan Compute Runtime

The primary driver for Vulkan GPGPU programming for this experiment is a queue-based GPU program dispatch runtime. The header `image.h` provides extensive primitives for allocating resources, defining upload and download paths, and modifying the execution queue. This architecture allows composable execution of GPU programs or 'compute shaders.'

The program is written in the C programming language, targeting a local Vulkan installation with only vendored dependencies (`stb_image.h`,`renderdoc_app.h`).

3.1.1 Execution Queue

The program was originally designed as an image processing tool, which is reflected in the internal API. The primary processing primitive is `img_t`, representing image types, and various GPU programming primitives (`img_gpu_t`,`img_gpu_pass_t`,`img_gpu_buffer_t`, `img_gpu_program_t`)